Stauxil

Privacy Request Inbox MVP

Codex-ready product and implementation design document

Product	Stauxil
Document type	Full MVP design document for implementation
Primary use	Hand-off to GPT Codex / coding agent and repo planning
Scope	Lightweight SaaS for receiving, verifying, managing, responding to, and documenting privacy requests
Working principle	Workflow system first; not a legal advice engine and not an automated all-system deletion platform

Core promise: Handle privacy requests without spreadsheets, missed deadlines, or messy email threads.

1. Overview

Product description

Stauxil is a lightweight SaaS that helps small businesses receive, verify, manage, respond to, and document privacy requests such as access, deletion, and correction requests.

What the product does

- Provides a public request form that customers can submit without signing in.
- Gives the business a private dashboard for reviewing and processing requests.
- Uses email-based verification so teams can confirm the requester is legitimate.
- Tracks deadlines, ownership, notes, emails, and a full activity trail.
- Supports basic exports so the business can retain evidence of what happened in each case.

Plain-English framing

Stauxil is a privacy request inbox for small businesses. It is a workflow and documentation layer, not a law firm, not a legal decision engine, and not an automatic delete-everything service.

2. Problem Statement

Small businesses increasingly need to handle privacy-related requests from customers, users, and clients, but most still process them through shared inboxes, spreadsheets, and ad hoc notes. That leads to missed deadlines, inconsistent verification, fragmented internal handling, and weak proof that the business responded appropriately.

- Requests arrive through multiple channels and are easy to miss.
- Owners do not always know who should handle a request next.
- Internal notes and customer-facing replies become scattered across email threads.
- Teams lack a case-level audit trail if they need to show what happened later.
- Enterprise privacy tools are often too expensive and too heavy for smaller teams.

3. Goals and Success Criteria

Primary goals

1. Let an external requester submit a privacy request through a branded public page.
2. Let the business verify the requester through email.
3. Let the business manage requests in a private dashboard.
4. Let the business send templated responses and keep a full activity trail.
5. Let the business export case history.

6. Keep the scope small enough for a solo developer to ship quickly on the current stack.

MVP success criteria

- A customer can sign up and configure a workspace.
- A public request page can accept a new request.
- The requester can verify their email.
- The internal team can process the request end-to-end inside the app.
- The team can send a final response and close the case.
- The request history is exportable and visible as an audit trail.

4. Non-Goals for the MVP

The MVP intentionally avoids broad compliance automation. The first version should be a reliable workflow tool, not a deep integration product or a legal expert system.

- No automatic discovery of personal data across third-party systems.
- No automatic deletion in external services.
- No legal advice or legal conclusions.
- No document or selfie identity verification.
- No SSO, advanced RBAC, or enterprise procurement features.
- No deep integrations with Stripe, HubSpot, Intercom, Gmail, or support tools in v1.
- No AI-generated legal policy drafting.

5. Users and Roles

Role	Description	Primary actions
External requester	Customer or user submitting a privacy request.	Submit request, verify email, receive follow-up emails
Workspace owner	Business owner or lead responsible for setup and oversight.	Configure workspace, assign requests, manage billing, close cases
Workspace member	Teammate helping process requests.	Review cases, add notes, send responses, update status

6. Core Workflows

Workflow A: submit request

7. Requester opens the public request form.
8. Requester selects access, delete, correct, or other request type.

9. Requester enters contact details, region, optional account reference, and request details.
10. System creates the request and sends verification email.

Workflow B: verify request

11. Requester clicks a magic link or enters a verification code.
12. System marks the request as verified.
13. An event is added to the audit trail and the dashboard updates immediately.

Workflow C: process request

14. Admin opens the case and reviews the request summary.
15. Admin assigns an owner and changes status to in progress.
16. Owner adds notes, requests more information if needed, and prepares a final response.
17. Owner sends the final email and closes the case.

Workflow D: export case history

18. Admin opens the request detail page.
19. Admin exports a printable case summary or CSV record.
20. Export includes request metadata, status changes, notes references, and email history.

7. Functional Requirements

7.1 Public intake form

- Public route supports unauthenticated request submission.
- Required fields: full name, email, request type, jurisdiction or region, request details.
- Optional field: account or customer reference.
- Workspace-configurable branding: company name, logo, intro copy, success message, allowed request types.

7.2 Verification

- Support email-based verification with either a magic link or a one-time code.
- Store verification state independently from request workflow status.
- Allow manual override by an internal user for edge cases.

7.3 Dashboard

- Display case ID, requester, request type, status, owner, created date, due date, and time remaining.
- Filters: status, owner, type, overdue only, and free-text search by case ID or requester email.
- Top-of-page summary cards: open requests, overdue requests, due this week, fulfilled this month.

7.4 Request detail page

- Show requester info, request summary, verification state, due date, internal notes, timeline, email history, and attachments.
- Actions: assign owner, update status, send verification email, add note, send email from template, mark complete, close case.

7.5 Deadline tracking

- Every request gets a due date based on workspace default SLA.
- Requests show due soon and overdue states clearly in the UI.
- Reminder emails should be triggered before and after the due date.

7.6 Templates and exports

- Support editable email templates for verification, acknowledgement, more-info-needed, completion, and denial.
- Provide CSV export of requests and a printable single-case summary.

8. Legal and Positioning Constraints

Stauxil must be presented as a workflow tool that helps teams manage privacy requests. The product should help with intake, verification, tracking, communication, and evidence. It should not promise legal outcomes or guaranteed compliance.

Use this language	Avoid this language
Manage privacy requests in one place.	Become instantly GDPR compliant.
Track deadlines, verification, and responses.	Guaranteed legal compliance.
Maintain an audit trail for each request.	Automated legal decision engine.

9. Technical Context and Existing Repo Fit

The current repository already contains the core SaaS starter pieces needed for a fast MVP build: Next.js App Router, Clerk authentication, Clerk pricing UI, Convex integration, and a placeholder authenticated homepage. The missing work is domain modeling, request workflow logic, and product-specific UI.

- Keep Clerk for authentication and billing UI.
- Use Convex as the persistence and realtime layer.
- Replace the current placeholder home page with the request inbox dashboard.
- Add a real Convex schema before building richer mutations and queries.

10. Architecture

Layer	Responsibilities
Next.js frontend	Public request form, verification page, dashboard, request detail pages, settings, templates, billing surfaces
Clerk	Authentication, user identity, plan-gated experiences, hosted pricing UI
Convex backend	Workspace data, requests, notes, events, templates, verification tokens, exports metadata
Email provider	Verification, acknowledgement, more-info-needed, completion, denial messages

11. Data Model

Create convex/schema.ts and define the following tables. The design favors explicit, easy-to-query records over highly abstracted domain objects.

Table	Key fields	Notes
workspaces	name, slug, ownerClerkUserId, supportEmail, defaultSlaDays, plan	Primary workspace record and branding/config container
workspaceMembers	workspaceId, clerkUserId, email, role	Links authenticated users to a workspace
requests	workspaceId, caseId, type, status, requesterEmail, dueAt, assignedToClerkUserId	Core case record
requestEvents	requestId, eventType, actorClerkUserId, metadata, createdAt	Immutable audit trail
requestNotes	requestId, authorClerkUserId, body, createdAt	Internal discussion and processing notes
emailTemplates	workspaceId, type, subject, body	Reusable internal templates
emailLogs	requestId, to, subject, body, sentAt	Outbound message history
verificationTokens	requestId, tokenHash, expiresAt, usedAt	Verification lifecycle storage

attachments (optional)	requestId, fileName, fileUrl, uploadedByClerkUserId	Reserve for uploaded evidence or exports

12. State Model

State area	Allowed values
Request status	new, awaiting_verification, verified, in_progress, waiting_on_requester, fulfilled, denied, closed
Verification status	pending, verified, failed, manual_review

Suggested status transitions

```
new -> awaiting_verification
awaiting_verification -> verified
verified -> in_progress
in_progress -> waiting_on_requester
waiting_on_requester -> in_progress
in_progress -> fulfilled
in_progress -> denied
fulfilled -> closed
denied -> closed
```

13. Routes

Route	Audience	Purpose
/request/[workspaceSlug]	Public	Branded intake form for new privacy requests
/verify	Public	Verification landing page for magic links or code entry
/request/success	Public	Confirmation page after successful submission
/	Authenticated	Dashboard / request inbox
/requests/[id]	Authenticated	Single request detail page
/templates	Authenticated	Workspace email template management
/settings	Authenticated	Workspace settings and branding

/billing	Authenticated	Billing page or redirect to Clerk pricing / portal

14. Component Plan

Area	Suggested components
Shared shell	AppShell, PageHeader, StatusBadge, DueDateBadge, EmptyState, ConfirmDialog
Public flow	RequestForm, RequestTypeSelect, VerificationCodeForm, RequestSuccessCard
Dashboard	DashboardStats, RequestFilters, RequestTable, RequestTableRow
Request detail	RequestSummaryCard, RequesterInfoCard, VerificationCard, RequestTimeline, NotesPanel, EmailHistoryPanel, AssignOwnerSelect, TemplateSendDialog
Settings	WorkspaceSettingsForm, BrandingSettingsForm, TemplateEditor

15. Convex Module Plan

The backend should be split into small, purpose-specific modules so Codex can implement, test, and extend them without a monolithic file.

```
convex/schema.ts
convex/workspaces.ts
convex/requests.ts
convex/requestEvents.ts
convex/requestNotes.ts
convex/emailTemplates.ts
convex/emailLogs.ts
convex/verification.ts
```

Required queries

- workspaces.getCurrentWorkspace
- requests.listForWorkspace
- requests.getById
- requests.getDashboardStats
- requestEvents.listForRequest
- requestNotes.listForRequest
- emailTemplates.listForWorkspace

Required mutations

- workspaces.createWorkspace
- workspaces.updateSettings
- requests.createPublicRequest
- requests.assignOwner
- requests.updateStatus
- requests.closeRequest
- requestNotes.addNote
- emailTemplates.upsertTemplate
- verification.createToken
- verification.verifyToken
- emailLogs.logEmailSent
- requestEvents.logEvent

16. Email Behavior

- Verification email sent immediately after request creation.
- Optional acknowledgement email after verification.
- Template-driven email for more information needed.
- Template-driven final completion or denial email.

Template variables

```
{{requesterName}}
{{companyName}}
{{caseId}}
{{requestType}}
{{dueDate}}
```

17. Security and Access Control

- Public submission route should be rate-limited or bot-protected.
- Verification tokens must be hashed at rest.
- Every authenticated query and mutation must validate workspace membership server-side.
- Never trust client-submitted workspace ownership or cross-workspace request IDs.
- Do not expose all requests across workspaces from a shared query.

18. Billing and Plan Gates

Plan	Core limits	Primary value

Starter	20 requests/month, 1 user, default branding	Low-friction entry tier for very small teams
Pro	100 requests/month, up to 5 users, exports, custom branding	Best default paid tier for small companies
Team	Unlimited requests and users, advanced exports, priority support	Higher-usage workspace with more operational control

MVP plan gating should focus on request volume, member count, export availability, and branding controls. Clerk pricing UI can remain the first billing surface.

19. Analytics

- Requests submitted by period
- Verification completion rate
- Average time to first response
- Average time to closure
- Overdue rate
- Template usage frequency

For v1, these metrics can be derived from request and event data rather than built as a separate analytics service.

20. UX Notes

- The dashboard should feel like an inbox: fast scanning, obvious status, minimal clutter.
- Trust and clarity matter more than decorative styling.
- Overdue warnings must be unmistakable without creating constant red-noise across the page.
- When there are no requests, the product should explain the workflow and point the owner to their public intake page.

21. Implementation Phases

Phase	Scope
Phase 1 - foundation	Add schema, workspace settings, dashboard shell, and request table backed by real data.
Phase 2 - core request flow	Build public request submission, case ID generation, verification flow, and detail page.
Phase 3 - operations	Add notes, timeline, status changes, owner assignment, dashboard

	stats, and email templates.
Phase 4 - monetization and polish	Apply billing gates, exports, branding settings, validation cleanup, and empty states.

22. Acceptance Criteria

21. A new user can sign in and create a workspace.
22. A workspace can configure company name, branding, support email, and default SLA.
23. A public request form saves requests to the correct workspace.
24. A verification flow updates request verification state.
25. The homepage shows request stats and a request table.
26. Clicking a request opens a full detail page.
27. Users can assign owner, update status, add notes, and see event history.
28. Users can send at least one templated email from the request page.
29. Requests can be exported as CSV or printable summary.
30. Workspace isolation is enforced in backend queries and mutations.
31. At least one paid feature is gated by billing plan.
32. The app passes lint, typecheck, and build validation.

23. Commands Codex Should Run

Use the repository scripts that already exist for validation. Convex development mode may also be needed during implementation.

```
npm install
npm run lint
npm run typecheck
npm run build
```

```
npx convex dev
```

24. File Plan for Codex

Files to modify

- app/page.tsx
- app/layout.tsx (only if navigation or app shell changes are needed)
- app/pricing/page.tsx or replace with /billing

- components/ConvexClientProvider.tsx only if provider adjustments become necessary
- convex/messages.ts may be removed or replaced

Files to add

- convex/schema.ts
- convex/workspaces.ts
- convex/requests.ts
- convex/requestEvents.ts
- convex/requestNotes.ts
- convex/emailTemplates.ts
- convex/emailLogs.ts
- convex/verification.ts
- app/request/[workspaceSlug]/page.tsx
- app/verify/page.tsx
- app/request/success/page.tsx
- app/requests/[id]/page.tsx
- app/settings/page.tsx
- app/templates/page.tsx
- supporting UI components under components/

25. Suggested AGENTS.md Guidance

```
# Stauxil MVP instructions

Build the MVP described in the design document.

Priorities:
1. Deliver working end-to-end request flow before polish.
2. Keep implementation simple and typed.
3. Enforce workspace isolation in every backend query and mutation.
4. Do not add external integrations unless required for core email verification.
5. Prefer small reusable components over large pages.
6. Use existing scripts for lint, typecheck, and build validation.

Required checks after changes:
- npm run lint
- npm run typecheck
- npm run build

Do not claim legal compliance in product copy. Use operational wording like:
- Manage privacy requests
- Track deadlines and verification
- Maintain an audit trail
```

26. Non-Technical Summary

Stauxil gives a company a branded form where customers can submit privacy requests, and a private workspace where the company can manage those requests from start to finish. It verifies the requester, tracks deadlines, stores notes and actions, helps teams send consistent replies, and preserves a case history so nothing gets lost.

27. Recommended Build Sequence

33. Workspace setup and branding
34. Public request form
35. Verification flow
36. Dashboard
37. Request detail page
38. Notes and status changes
39. Template sending
40. Export

That order produces a real, sellable MVP as quickly as possible while keeping the architecture straightforward for a solo developer.